

# Learning from Click-Logs

*A two-stage retrieve-then-rank system for MIND and EB-NeRD*

**Chaitanya Shah** — CS4.406 Information Retrieval & Extraction, Assignment 2 — 20 September 2026

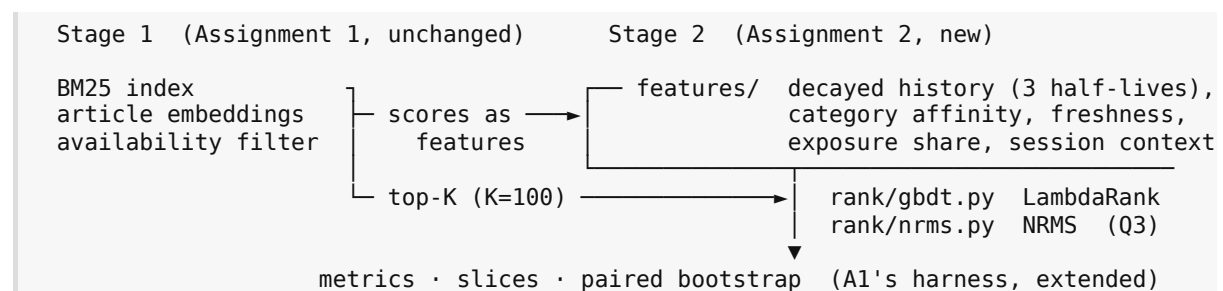
**Repository:** [https://github.com/ChaitanyaShah21/IRE\\_A2](https://github.com/ChaitanyaShah21/IRE_A2)

**Supporting material:** decision log ARCHITECTURE.md (D1-D41); results as CSV and JSON under reports/; 326 tests. Every figure below traces to reports/NUMBERS.md, which records the command that produced it.

**Summary.** Assignment 1 established that lexical and semantic retrieval contain no notion of time, while 92.7% (MIND) and 93.5% (EB-NeRD) of clicks fall on fresh articles. This system adds behavioural features and a trained re-ranker above that retrieval stage. On EB-NeRD it raises test AUC from 0.5457 to 0.7428, and the leaderboard confirms the level at 0.7397 against A1's 0.5396; on MIND it produces no useful gain (0.6181 against 0.6191), for reasons §6 quantifies.

## 1. System overview

Assignment 1's scorers are retained as **features** rather than replaced. `cos_inf`, the user profile pooled with an infinite half-life, reproduces A1's semantic score exactly, and `bm25` is A1's lexical score. The "before re-ranking" column of every table in §3 is therefore one input column of the "after" system, so the comparison is between two orderings of one candidate list rather than between two unrelated systems.



**Two candidate sets are kept separate throughout (D37).** Both competitions supply a candidate list per impression (`article_ids_inview`, `impressions`) and score our ordering of it. Q2 additionally requires retrieval of top-K from the whole corpus. These are different operations on different candidate sets; each table below states which it reports.

## 2. Behavioural features and the behaviour-window boundary (Q1)

| Feature                                                                                                          | MIND                     | EB-NeRD           |
|------------------------------------------------------------------------------------------------------------------|--------------------------|-------------------|
| <code>cos_short</code> , <code>cos_medium</code> , <code>cos_inf</code> — decayed user profile · candidate (D35) | ✓ (3 / 10 / ∞ positions) | ✓ (1 d / 7 d / ∞) |
| <code>cat_share_short</code> , <code>_medium</code> , <code>_inf</code> — recency-weighted category affinity     | ✓                        | ✓                 |
| <code>bm25</code> — A1's lexical score, N = 100 (D31)                                                            | ✓                        | ✓                 |
| <code>freshness_hours</code> — <code>T - min(published_time, first_seen)</code> (D34c)                           | ✓ (first_seen only)      | ✓                 |
| <code>exposure_share_1h</code> , <code>_24h</code> — label-free share, strictly before T (D34b)                  | ✓                        | ✓                 |
| <code>hours_since_last_click</code> (D35)                                                                        | —                        | ✓                 |
| <code>session_position</code> , <code>minutes_since_session_start</code> , <code>device_type</code> (D34a)       | —                        | ✓                 |

**The feature set is an allowlist, and the builder asserts it:** output columns must equal keys + label + the list above. The reason is concrete. `total_pageviews` and `total_inviews` are present in `articles.parquet` and are whole-dataset aggregates containing the future, so any builder that iterates over article columns absorbs them without raising an error.

**Boundary enforcement is tested as two properties rather than a checklist.** *Future deletion*: delete every impression after  $\tau$ ; features at or before  $\tau$  must be byte-identical. *Label blindness*: shuffle the clicks; every feature must be unchanged. New features are covered automatically. Mutation testing planted four realistic forward leaks in production code and all four were caught — but only once the check was extended to MIND, because on EB-NeRD `published_time` almost always wins the `min`, leaving the `first_seen` path unexercised.

### Three data hazards, measured rather than assumed.

- EB-NeRD's article catalogue extends a month beyond the test week (2,148 articles are published after the training week ends), so no catalogue-wide statistic is used as a feature.
- `session_id` is not unique across users (1,032 shared ids). Sessions are keyed (user, session): keyed by id alone, the longest session spans 14 days; keyed correctly, 29 minutes.
- 1,810 candidate rows are shown *before* their `published_time`, so freshness is measured from the earliest evidence of existence rather than from the catalogue timestamp.

## 3. Two-stage retrieve-then-rank (Q2)

**Model (D36).** LightGBM with the `lambdarank` objective, which is scored only on ordering each impression's clicked candidates above its unclicked ones, weighted by the nDCG impact of a swap. Training uses the train split; the tree count is chosen on validation nDCG@10; all reported numbers come from the local test split, which selected nothing. Hyperparameters were fixed before any result was observed. LightGBM infers impression boundaries from group *sizes* alone, so shuffled rows train successfully and rank randomly; `group_sizes` derives those boundaries from impression ids and refuses non-contiguous input.

### 3.1 Supplied candidate lists — the task both leaderboards score

| System                         | MIND AUC                 | MIND nDCG@10  | EB-NeRD AUC              | EB-NeRD nDCG@10 |
|--------------------------------|--------------------------|---------------|--------------------------|-----------------|
| random                         | 0.4982                   | 0.2756        | 0.5007                   | 0.4288          |
| bm25 (A1)                      | 0.5641                   | 0.3265        | 0.5041                   | 0.4305          |
| semantic (A1) — before         | 0.6097                   | 0.3588        | 0.5457                   | 0.4585          |
| <b>LambdaRank (A2) — after</b> | <b>0.6144</b>            | <b>0.3636</b> | <b>0.7428</b>            | <b>0.6250</b>   |
| paired gain, 95% CI            | +0.0048 [0.0012, 0.0080] | +0.0048       | +0.1971 [0.1953, 0.1989] | +0.1665         |

Both AUC intervals exclude zero, and the asymmetry between datasets is itself a result. Feature importances locate it: on EB-NeRD, freshness (38%) and exposure share (34%) account for 73% of the model's gain; on MIND, `cos_inf` leads at 27% and the model saturates after 18 trees, with validation nDCG@10 flat from roughly 20 to 300 trees while training nDCG@10 climbs from 0.396 to 0.457. Single-feature AUCs on MIND place freshness at 0.534 and exposure at 0.533 against similarity's 0.610. MIND publishes no article timestamps and no history timestamps, so freshness must be inferred from first appearance in a sampled log and recency can use only list position. The temporal signal

contributes where the data records time; the two datasets' decay features are consequently not comparable with one another.

### 3.2 Retrieved top-K ( $K = 100$ )

A1's semantic generator retrieves 100 candidates from the corpus and the same model re-ranks them (reports/retrieved\_regime\*\_test\_k100.csv). Recall@100 is 2.6% (MIND) and 2.5% (EB-NeRD), so ranking metrics are defined on 798 and 4,704 impressions respectively. Within those, MRR improves from 0.070 to 0.260 (MIND) and from 0.052 to 0.567 (EB-NeRD). Recall@K bounds the result: re-ranking cannot recover an impression whose clicked article never entered the retrieved set, which is why both competitions score a supplied list.

EB-NeRD's AUC in this regime ( $0.508 \rightarrow 0.968$ ) is **not** evidence of a better system than §3.1's 0.743. The 99 negatives are drawn from a 125,541-article corpus by text similarity, so most were not in circulation that day and freshness separates them almost perfectly. Easy negatives inflate AUC; the supplied-list figure is the one to rely on.

## 4. Baseline reproduced, then improved (Q3)

**Baseline (D39): NRMS** (Wu et al., 2019) — multi-head self-attention over the user's clicked articles, additive attention to pool them, dot-product scoring against the candidate, trained with one click against four same-impression negatives.

**Documented deviation.** The news encoder is a learned projection of the 384-dimension multilingual sentence embeddings computed in Assignment 1, rather than word-level attention over title tokens. The user encoder, the training objective and the scoring are the paper's.

**What a word-level encoder would change.** Word-level attention learns which *terms* in a headline matter to a given user, a resolution a fixed sentence vector cannot express, and published NRMS results use it with GPU training over full datasets; absolute scores here are correspondingly lower. It was not attempted because it requires a tokeniser, a vocabulary, pre-trained word vectors and several hours of CPU training per arm, and the ablation needs four arms. The substitution shifts the level of every arm equally, so the ablation below — what Q3.3 and Q3.4 grade — is unaffected. This section is a controlled ablation around a reproduced architecture, not a claim about NRMS's ceiling.

**The change under test (D40): a time term.** NRMS receives no temporal input. A small network over (log freshness, exposure share 1 h, exposure share 24 h) is added to the click score, with its output layer initialised to zero so that training begins at exactly the baseline. The change was selected from Assignment 1's measurements before any NRMS result existed.

**Ablation and significance.** Four arms, identical except for the time inputs, each trained on the same seeded 60,000 impressions (D41) and evaluated on the entire test split. Paired bootstrap, 1,000 resamples.

| Arm                                   | MIND AUC | paired $\Delta$            | EB-NeRD AUC   | paired $\Delta$                 |
|---------------------------------------|----------|----------------------------|---------------|---------------------------------|
| nrms (reproduced baseline)            | 0.6146   | —                          | 0.5874        | —                               |
| + freshness                           | 0.6159   | +0.0013 [0.0004, 0.0022]   | 0.7014        | +0.1141 [0.1126, 0.1155]        |
| + exposure share                      | 0.6122   | −0.0024 [−0.0032, −0.0014] | 0.6244        | +0.0370 [0.0359, 0.0380]        |
| <b>+ both — pre-registered change</b> | 0.6144   | −0.0002 [−0.0011, 0.0008]  | <b>0.6738</b> | <b>+0.0864 [0.0852, 0.0877]</b> |

On EB-NeRD the interval excludes zero by a wide margin (+0.0864 AUC, +0.0628 MRR). On MIND the pre-registered arm is indistinguishable from the baseline on AUC, although it improves MRR (+0.0053 [0.0038, 0.0067]).

The ablation supports three statements that a single "improved" number could not. It **locates** the gain in freshness, and shows exposure share contributing alone (+0.0370) but subtracting when combined, the two being correlated. It **reproduces the §3 ordering in a different model class**, since the LambdaRank importances also rank freshness first on EB-NeRD and find exposure inverse on MIND. And it **bounds where the change transfers**: MIND, lacking publication and history timestamps, has less temporal signal to add.

**Two limits, one of them tested.** The arms train for 3 epochs with validation AUC still rising, so absolute figures are undertrained. Re-running all four MIND arms for **6 epochs** (nrms\_ablation\_mind\_test\_6epochs.csv) lifts every arm  $\sim +0.0035$  AUC and changes no conclusion: baseline  $0.6146 \rightarrow 0.6180$ , + both  $-0.0002 \rightarrow -0.0001$  (still spanning zero), + freshness  $+0.0013 \rightarrow +0.0012$ , + exposure  $-0.0024 \rightarrow -0.0020$ , with the last validation gain down to  $+0.0007$ . **The MIND null is a property of the dataset, not of the budget.** The table keeps the 3-epoch arms because EB-NeRD's share that budget. Second, + freshness alone outperforms the pre-registered + both; that is reported as an ablation observation rather than promoted after the fact, the error recorded as Finding 5 in the decision log.

## 5. Serving characteristics (Q4)

Measured per single user request ( $K = 100$ ), single process, with no other work running; training was suspended for the measurement, because a latency figure taken on a contended machine has already had to be withdrawn once in this project.

| p50 / p99 per stage                                                   | MIND                              | EB-NeRD                           |
|-----------------------------------------------------------------------|-----------------------------------|-----------------------------------|
| retrieve top-100                                                      | 9.0 / 23.7 ms                     | 9.6 / 22.2 ms                     |
| <b>build features</b>                                                 | <b>223.0 / 348.0 ms</b>           | <b>439.0 / 2,352 ms</b>           |
| score 100 candidates                                                  | 3.2 / 9.0 ms                      | 4.3 / 6.9 ms                      |
| <b>total</b>                                                          | <b>237 / 360 ms</b>               | <b>452 / 2,369 ms</b>             |
| footprint · throughput · cost per 1,000 queries · cores for 1,000 QPS | 283 MB · 4.2 q/s · \$0.0027 · 239 | 475 MB · 1.5 q/s · \$0.0077 · 689 |
| p99 < 100 ms SLA                                                      | not met                           | not met                           |

Footprint detail: embeddings 100 / 193 MB, exposure index 71 / 123 MB, user vectors 77 / 61 MB, BM25 19 / 22 MB, and EB-NeRD's session table 44 MB. Start-up (context build) is 5.2 s locally and 137 s at leaderboard scale. Cost assumes \$0.04 per vCPU-hour.

**Little's law check.** With one request in flight, throughput  $\leq 1/\text{latency}$ : MIND's  $R = 0.237$  s bounds  $X$  at 4.2 q/s, which equals the measured per-core throughput.

**The SLA failure is architectural rather than algorithmic.** Retrieval (9 ms) and scoring (3–4 ms) are within budget; feature assembly is 93–97% of the request. Per-user profile construction accounts for only 13.5% (MIND) and 16.8% (EB-NeRD) of that stage, so precomputing profiles is necessary but insufficient. The remainder is per-call overhead: the same builder processes  $\sim 11,500$  rows/s in bulk and  $\sim 450$  rows/s one request at a time, because it joins whole-dataset tables — a 1.2M-row session table, 125k articles, a 65k-entry category map — to answer a 100-row question.

**Projected remedy** (a projection, not a measurement): precompute user profiles into a key-value store (−30 ms), replace per-request joins with array lookups keyed by article row, and cache article features by (article, hour). Stages 1 and 3 total ~13 ms, so a ~20 ms lookup path would place p99 near 40 ms and cost near \$0.0003 per 1,000 queries. In batch the same system is efficient — 206M rows scored in 155 minutes, ~22,000 rows/s — so it is a strong batch scorer and a weak online one.

## 6. Extended evaluation with slices (Q5)

All metrics on the supplied-list pipeline, local test split, 1,000-resample percentile intervals. Slices follow D26: cold means five or fewer history articles; head and tail are by exposure, where an impression counts as head only if every clicked article is in the head set. Mixed impressions (2,922 MIND, 243 EB-NeRD) belong to neither and are counted.

| Slice                                             | MIND: semantic → LambdaRank                                  | EB-NeRD: semantic → LambdaRank                             |
|---------------------------------------------------|--------------------------------------------------------------|------------------------------------------------------------|
| all                                               | 0.6097 → 0.6144, <b>+0.0048</b><br><b>[0.0012, 0.0080]</b>   | 0.5457 → 0.7428, <b>+0.1971</b><br><b>[0.1953, 0.1989]</b> |
| cold (n = 3,904 / 680)                            | 0.5642 → 0.5529, <b>−0.0113</b>                              | 0.5538 → 0.7348, <b>+0.1810</b>                            |
| warm (n = 18,043 / 186,041)                       | 0.6195 → 0.6277, +0.0083                                     | 0.5457 → 0.7428, +0.1971                                   |
| head-exposure (n = 13,108 / 98,719)               | 0.5955 → 0.5828, <b>−0.0127</b><br><b>[−0.0176, −0.0079]</b> | 0.5565 → 0.7745, +0.2179                                   |
| tail-exposure (n = 5,917 / 87,759)                | 0.6319 → 0.6671, <b>+0.0353</b>                              | 0.5334 → 0.7076, +0.1741                                   |
| diversity (category) · novelty · coverage, K = 10 | 0.817 → 0.735 · 17.34 → 17.69 · 0.0283 → 0.0318              | 0.799 → 0.803 · 18.87 → 19.06 · 0.0260 → 0.0247            |

**EB-NeRD's gain is uniform across slices**, largest on head-exposure (+0.218) and smallest on tail (+0.174) — the expected ordering for a model that uses popularity, and therefore a consistency check rather than a surprise.

**MIND's aggregate +0.0048 conceals two sign reversals**, which is the purpose of slicing. The model is worse on cold users (−0.0113) and on head-exposure impressions (−0.0127 [−0.0176, −0.0079]), better on warm users (+0.0083) and substantially better on tail-exposure impressions (+0.0353). The mechanism is measured rather than inferred. Within a MIND impression, a candidate's exposure share predicts clicks in the **inverse** direction (AUC 0.476 taken positively), while across impressions clicked rows carry a *higher* mean exposure share than unclicked rows (0.384 against 0.271) — a trend that reverses inside groups. A MIND rack consists mainly of the site's standard rotation plus a few user-specific items, and the click typically lands on the less-shown item. The model learns to demote widely-shown articles, which helps when the clicked article is obscure and harms when it is popular.

**Diversity is not free.** On MIND the re-ranker trades category diversity (0.817 → 0.735) for novelty and coverage; on EB-NeRD it costs coverage (0.0260 → 0.0247) while raising novelty. Coverage carries no confidence interval (D27): the bootstrap is biased downward for a union statistic, so the raw spread is reported in the CSV under its own columns.

## 7. Anti-gaming (Q9)

**Leakage test.** The two properties described in §2, mutation-verified against four planted leaks. A quarantined future-looking feature fails the same check, which demonstrates that the check can detect a leak.

**Metrics with and without serving-unavailable features.** The shipped model is trained twice: on the allowlist, and on the allowlist plus `read_time`,

scroll\_percentage, total\_pageviews and future\_exposure\_share\_24h.

| AUC                                                        | MIND (21,947)                     | EB-NeRD (186,721)               |
|------------------------------------------------------------|-----------------------------------|---------------------------------|
| honest — shipped, and the source of both leaderboard files | 0.6144 [0.6104, 0.6185]           | 0.7428 [0.7415, 0.7441]         |
| + serving-unavailable                                      | 0.5994 [0.5952, 0.6033]           | 0.7590 [0.7578, 0.7602]         |
| paired difference                                          | <b>-0.0150 [-0.0181, -0.0118]</b> | <b>+0.0162 [0.0153, 0.0172]</b> |

On EB-NeRD the advantage from unavailable features is +0.0162 AUC: statistically clear, but less than a tenth of what the honest behavioural features contribute (+0.197). It is carried by `total_pageviews` (17% of the leaky model's gain) and `future_exposure_share_24h` (16%).

**read\_time and scroll\_percentage contribute nothing, for a structural reason.**

EB-NeRD records both per impression, so they are constant across a rack, and a ranking objective compares candidates only within a rack. Neither appears among the leaky model's five most important features. The exploitable leak is the one that varies per candidate, which is why a per-article whole-dataset aggregate is the largest offender here.

**MIND cannot price the leak, for a mechanical reason.** MIND publishes no dwell or pageview fields, leaving only the forward exposure window. MIND's log ends 2019-11-15 23:58, so that window spans 37–138 h in training but only 5.7–11.4 h at test time, inflating the feature's mean by a factor of 3.2 (0.075 → 0.241). The leaky model relied on it (118 trees against 18, and 15.7% of its gain) and then received a differently distributed input at test time. EB-NeRD, where all four columns exist and three are not window-dependent, is where the price is measurable.

## 8. Behaviour at 10× scale

The basis for this section is a completed run at the largest scale available: 205,925,868 candidate rows (EB-NeRD) and 93,115,001 (MIND) were featurised and scored locally in 155 and 26 minutes, at a 10.6 GB peak on an 11 GB machine.

Three properties made that possible. The exploded candidate list is never materialised as strings (206M pairs cost ~10 GB as strings, 1.4 GB as one sorted `int64` key each). A deep slice of a lazy frame is not free once a row index is attached — one 5,000-row chunk at offset 5M cost 297 s, fixed by writing the prepared frame to Parquet first. And work is partitioned by user, so per-user computation (~3.4 ms) happens once rather than once per chunk containing that user.

At 10× (135M impressions, ~2 G candidate rows) the constraints bind in this order:

1. **The exposure index.** 2 G `int64` keys is 16 GB and no longer fits. It must become time-bucketed shards, or a count keyed by (article, hour): 125k articles × 168 hours is 21M cells, a bounded structure.
2. **Per-user feature construction.** Linear and currently unamortised: ~8 M users × 3.4 ms ≈ 7.5 h. A production system computes profiles in a nightly batch, since a profile depends on history rather than on the request.
3. **Wall clock.** ~26 h on one machine. The work is parallel by user group and requires no global shuffle, so this is a horizontal-scaling problem rather than an algorithmic one.

The model (322 trees, 1.6 MB), the embeddings (190 MB) and the BM25 index do not grow with traffic; the log-derived features do.

## 9. Leaderboard results

|                                 | Assignment 1 system | This system   | Offline test AUC |
|---------------------------------|---------------------|---------------|------------------|
| EB-NeRD (RecSys 2024 Challenge) | 0.5396              | <b>0.7397</b> | 0.7428           |
| MIND                            | 0.6191              | <b>0.6181</b> | 0.6144           |

EB-NeRD improves by +0.2001 AUC, and the offline harness predicted that level to within 0.0031 on 13.5M unseen impressions scored by the organisers' own program. That agreement is the principal evidence that the offline figures in this note are meaningful: the leaderboard returns a single number and no diagnostics, whereas the slices, ablations and intervals reported above are observable only offline.

On MIND the two systems are indistinguishable ( $-0.0010$ ) despite an offline gain of +0.0048. The offline *level* transferred on both datasets (within 0.004); the offline *delta* transferred only where the signal was large. §6 had already identified the cause: the MIND gain conceals a  $-0.0127$  loss on head-exposure impressions, which dominate the leaderboard week.

## 10. Limitations and next steps

- **MIND requires rack-relative popularity features.** Absolute exposure is learned in the inverse direction and damages head-exposure impressions; normalising within an impression ( $\text{exposure} \div \text{rack maximum}$ ) removes a threshold ambiguity trees cannot resolve.
- **Cold users need a separate path:** on MIND they lose 0.0113 AUC, their decayed-profile features being null while the remaining temporal features are weak.
- **NRMS uses a substituted news encoder** (§4); a word-level encoder with full-data GPU training would raise every arm.
- **Online serving requires restructuring rather than tuning** (§5).

## 11. Reproduction and authorship

`README.md` documents a one-command rebuild and the scripts that regenerate every table above. `AI_USAGE.md` marks each file's authorship, `reports/ai_transcripts/` holds the prompt log for all 16 sessions (Q7.4), and leaderboard screenshots are in `reports/figures/`. The `codabench/` tooling exists to obtain a score on a competition whose hosted workers were retired; the organisers' scoring program, hidden reference data and metrics are unmodified.